

Probabilistic Planning, Deterministic Execution: Why Autonomous Agents Demand Transport-Layer Action Governance

An Architectural Blueprint for Fail-Closed Tool Interception, Sequence Verification, and Cryptographic Evidence Chains

SASKI Institute PBC

Executive Summary

The enterprise deployment of artificial intelligence has transitioned from passive conversational interfaces to autonomous, agentic systems. Equipped with Model Context Protocol (MCP) tool bindings, enterprise APIs, and direct system access, modern AI agents autonomously formulate multi-step plans, invoke tools, modify databases, and command physical infrastructure at machine speed.

This expansion of operational agency introduces an existential governance crisis. Large language models (LLMs) operate over continuous vector spaces where behavior is probabilistic, non-deterministic, and context-sensitive. Yet they are increasingly placed in command of physical environments, enterprise cloud infrastructure, financial transactions, and mission-critical workflows that strictly require discrete, mathematically verifiable compliance guarantees.

The prevailing industry response has attempted to solve agent safety through prompt engineering, in-context system instructions, reinforcement learning, and secondary supervisor models ("LLM-as-a-judge"). A growing body of empirical research confirms that in-model, probabilistic safeguards cannot provide deterministic security lower bounds. When autonomous agents are exposed to Sequential Tool Attack Chaining (STAC), prompt injections embedded in untrusted downstream data, or subtle instruction drift, internal safety guardrails fail silently and reliably.

Agentic AI does not have a model-alignment problem; it has an execution architecture problem. Relying on a probabilistic model to decide whether its own planned action is authorized, compliant, or safe conflates cognition with enforcement.

This paper presents the architectural blueprint for **SASKI Agentic**: an independent, transport-layer enforcement gateway designed to decouple probabilistic planning from deterministic execution. By intercepting tool calls at the execution boundary across five domain-blind invariants, enforcing stateful sequence rules, and emitting tamper-evident cryptographic receipt chains anchored to immutable storage, SASKI Agentic converts autonomous tool invocation into a secure, auditable, and commercially insurable discipline.

The Core Thesis: Probabilistic Planning + Deterministic Governance

To build defensible agentic systems, enterprise architects must establish a clear structural boundary between two operational domains:

- **Probabilistic Planning:** A foundation model evaluates context, generates creative strategies, reasons across unstructured inputs, and proposes a structured sequence of tool calls. The model is an optimization engine for natural language and task synthesis.
- **Deterministic Governance:** An external, independent gateway intercepts every proposed tool invocation before it reaches downstream APIs, databases, or physical equipment. It evaluates the call against an explicit, customer-owned rulebook and produces an immutable, mathematically reproducible verdict.

***The Governing Axiom:** Probabilistic components may escalate, never authorize.*

A language model or statistical classifier may flag a proposed action as anomalous, risky, or ambiguous and route it to a human supervisor for manual review. However, a probabilistic component must **never** be granted the authority to grant an execution permit on its own judgment. The deterministic gateway remains the sole component capable of producing an allow verdict. A decision is genuine evidence only if it can be re-run and reproduced identically; an authored rulebook evaluated against recorded inputs is fully reproducible, whereas a foundation model whose weights drift across vendor updates is not.

Deterministic governance does not imply that policy definition is infallible. It guarantees that once an enterprise rule is declared, its enforcement is executed reproducibly, fail-closed, and completely independent of model variance.

1. The Failure of In-Model Safety in Agentic Systems

Autonomous agents dynamically interact with environments through cyclic execution loops: observe, plan, invoke tool, inspect tool output, and repeat. In-model safety mechanisms degrade across three distinct failure vectors in this loop:

Sequential Tool Attack Chaining (STAC)

Traditional guardrails evaluate inputs and tool calls in isolation. In real-world enterprise deployments, catastrophic harm rarely requires an explicitly malicious single command.

The STAC framework demonstrates that adversaries can systematically bypass frontier model alignments by decomposing malicious tasks into an ordered sequence of individually benign tool calls. Empirical evaluations across state-of-the-art agent frameworks report an average final attack success rate of **91.2%** under sequential chaining. If an agent evaluates each step in isolation—such as reading an access log, altering a routing parameter, and then resetting a connection—a stateless guardrail permits each call, remaining blind to the destructive objective achieved by the composition.

Indirect Injection and Poisoned Device Responses

When an agent reads data from external systems (e.g., an email body, a customer support ticket, a sensor payload, or a network status query), that data enters the model's active reasoning context. Malicious or malformed inputs can manipulate the agent into executing secondary, unauthorized actions. In-model filters cannot reliably distinguish between trusted operational directives and untrusted external payloads.

Constraint Priority Inversion and Tool Suppression

To force models to interact with APIs, developers rely on grammar-constrained decoding (e.g., forcing output into a JSON schema). However, when foundation models face simultaneous structured output constraints and tool-use tasks, syntactic compliance supersedes semantic veracity—a phenomenon known as *Constraint Priority Inversion (CPI)*.

Grammar masks render necessary error-handling or tool-call tokens unreachable during token generation. Under schema pressure without an explicit abstention path, models fabricate missing arguments or silently fail to execute underlying tool invocations while outputting syntactically valid payloads.

2. Complete Mediation & The Standing Service Topology

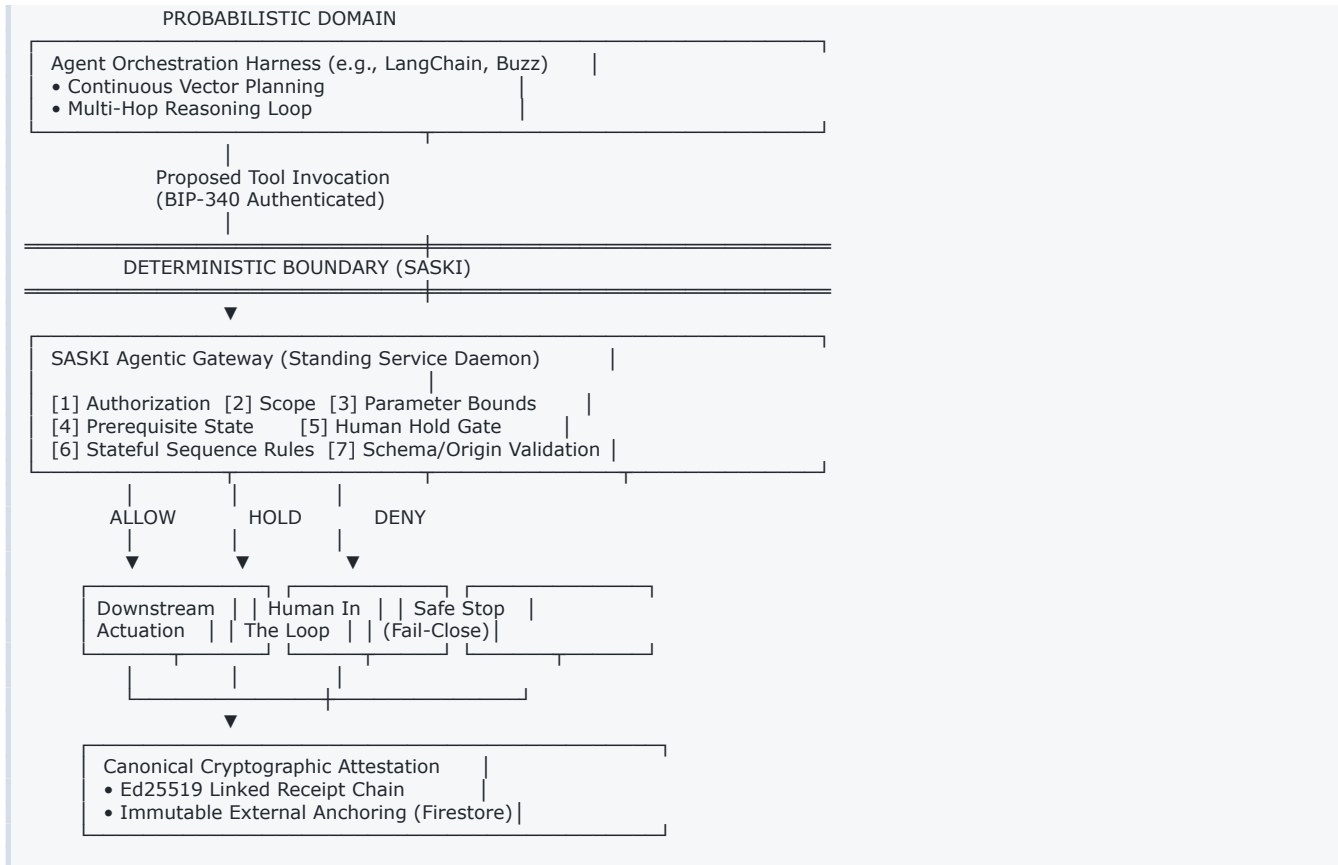
A governance gateway must enforce **complete mediation**: an autonomous agent must be structurally incapable of bypassing the control layer to interact with tools directly.

Process Isolation Over Subprocess Spawning

Early agent architectures embedded governance as a client library or spawned the gateway as a subprocess launched by the agent harness itself. This topology suffers from a fundamental security flaw: the agent process can inspect its own environment, spoof its identity, manipulate local configuration files, or bypass the wrapper to open direct network sockets to downstream devices.

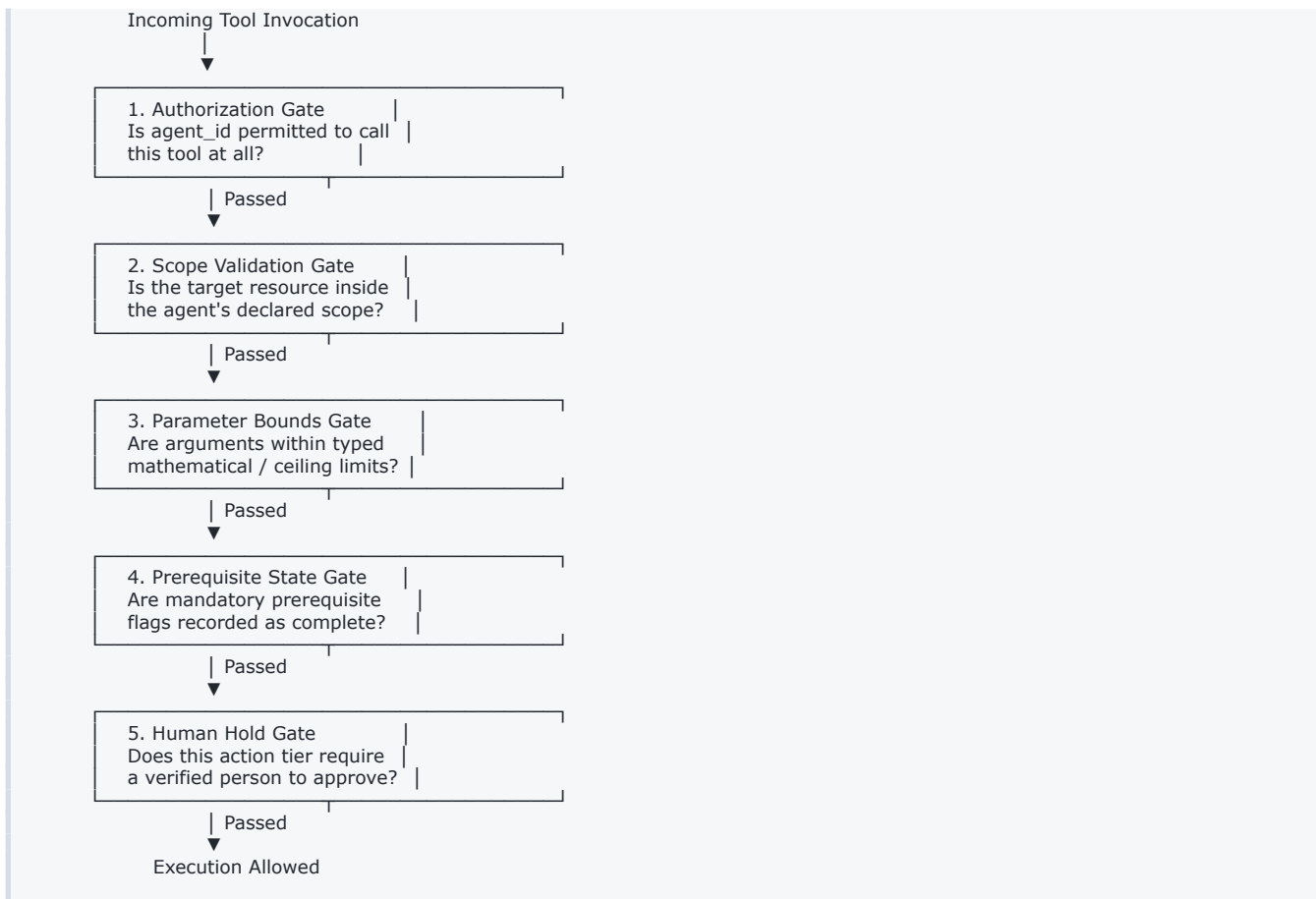
SASKI Agentic enforces complete mediation by operating as an independent **standing service**:

- **Dedicated Service Perimeter:** The gateway runs as an external daemon over loopback TLS/HTTP, strictly separated from the agent execution harness. Downstream tool and device credentials are held exclusively by the gateway, never exposed to the agent process or its environment variables.
- **Cryptographic Agent Identity (BIP-340 Schnorr):** When an agent connects to the gateway, it cannot simply assert an identity string. The gateway issues a cryptographically secure, single-use challenge over a bounded 60-second replay window. The agent must sign this challenge using Nostr BIP-340 Schnorr signatures over its dedicated keypair.
- **Tenant-Bound Connection Pooling:** To support multi-tenant enterprise deployments without risk of cross-talk, downstream adapter connections bind strictly to verified tenant identifiers upon checkout. Any checkout attempt across mismatched tenant contexts fails closed without disturbing the existing connection.



3. The Five Action-Boundary Invariants

When a tool call hits the gateway, it is evaluated across five discrete, domain-blind checks. The checker does not perform semantic reasoning; it compares structured arguments against a signed, versioned operational rulebook.



1. Authorization

Verifies whether the authenticated `agent_id` or declared `agent_role` is explicitly permitted to invoke the tool. If an agent attempts to invoke an undeclared tool or a capability outside its permission set, the gateway halts execution fail-closed with an `UNKNOWN_TOOL` or `AUTHORIZATION_DENIED` verdict, preventing any downstream socket from opening.

2. Scope

Constrains an agent's authority to specific physical or logical resources. For instance, an agent authorized to control audiovisual equipment or environmental controls in "Conference Room A" is strictly blocked if it attempts to pass parameters addressing "Conference Room B" or an enterprise-wide broadcast zone (`SCOPE_DENIED`).

3. Parameter Bounds

Enforces strict mathematical floors, ceilings, and schema boundaries on tool arguments. A financial agent cannot execute a transfer exceeding a declared single-transaction threshold; an automated facilities agent cannot set HVAC or power parameters outside acceptable operating envelopes. Missing required parameters or type violations halt execution immediately.

4. Prerequisite State

Enforces workflow dependencies by confirming that mandatory operational conditions have been recorded prior to execution. An agent cannot execute an actuation tool (e.g., `start_conference` or

dispatch_remediation) unless prior state flags (e.g., system_initialized, identity_confirmed) are cryptographically recorded as satisfied.

***The Boundary of Responsibility:** SASKI deterministically checks whether a prerequisite is recorded as complete by an authorized entity. It never assumes professional liability by judging whether completing that prerequisite was clinically, financially, or operationally correct.*

5. Human Hold

Classifies high-consequence operations (such as power cycling an entire building, modifying firewall configurations, or executing large funds transfers) into a tier requiring explicit human sign-off. The gateway pauses the workflow, enters a managed pending state, and dispatches an approval card containing the full execution context to an external human-in-the-loop surface (e.g., Slack or CLI). Execution resumes only upon receiving a cryptographically verified decision from a named human approver.

4. The Four Deterministic Verdicts

The gateway evaluates each tool call and resolves to one of four discrete verdicts:

- **ALLOW:** The invocation satisfies all five invariants. The gateway forwards the call to the downstream adapter, generates an Ed25519-signed receipt, and records the event into the execution chain.
- **DENY:** The invocation violates a boundary, attempts an undeclared action, or experiences a runtime fault. The gateway executes a safe stop, halts forwarding, and signs a denial receipt citing the specific failure reason. Enforcement defaults to fail-closed across all consequential tools.
- **MODIFY (Redact-and-Forward):** Used when personally identifiable information (PII), confidential client data, or secrets are detected in tool arguments or responses. SASKI redacts or sanitizes the payload at the transport layer before forwarding, recording the modification event in the audit trail.
- **HOLD:** The proposed action is permitted by policy but classified as high-consequence. The gateway suppresses immediate forwarding, emits a signed hold receipt with a unique hold_id, and routes the decision package to a human approver.

5. Multi-Hop Sequence Reasoning & Temporal Enforcement

To neutralize multi-turn exploits such as STAC, governance must extend beyond single-hop parameter validation to multi-hop trajectory tracking.

Stateless guardrails suffer from "structural amnesia," seeing only the immediate tool invocation. In contrast, SASKI Agentic enforces stateful sequence rules defined directly in the customer rulebook:

- **Explicit Trajectory Invariants:** Sequence rules formalize prohibited state transitions. For example, in an enterprise building environment, a rule may dictate that disabling a security system followed by an immediate access reconfiguration must result in an immediate DENY. Similarly, powering down life-safety or environmental systems while occupancy state is non-zero results in an absolute block.
- **Negative Constraint Design:** Multi-step sequence rules are strictly designed to **deny** or **hold**; they never issue an autonomous allow. This ensures that complex trajectories cannot engineer an unmonitored execution shortcut.
- **Device Response Validation:** Downstream tool and device responses are subjected to deterministic schema, origin, and value-range validation before being returned to the agent harness. By verifying that an adapter response matches its declared type structure and originated from the authentic device socket, the gateway strips out potential prompt-injection vectors before they can re-enter the model's reasoning context.

6. Cryptographic Attestation & External Evidence Anchoring

For an enterprise, an autonomous action that cannot be proven to an underwriter, regulator, or court is an uninsurable liability. SASKI Agentic guarantees that every consequential decision produces an unbroken, mathematically verifiable evidence chain.

Canonical Receipt Chains

Every consequential decision emits an **Ed25519 cryptographic receipt** canonicalized under RFC 8785 (JSON Canonicalization Scheme). The receipt payload binds:

- The authenticated agent_id (Nostr BIP-340 public key).
- The originating workflow and turn identifiers (workflow_id, origin_event_id).
- A cryptographic digest of the parameters (sha256_hex over canonical bytes). Raw arguments containing sensitive data or PII are never persisted in cleartext in the receipt.
- The active policy_version and the exact rulebook_digest in force.
- The deterministic verdict (allow, deny, modify, hold).
- The named human decision-maker (decided_by), if routed through a hold.
- A cryptographic back-link to the previous receipt (prev_receipt_id), forming an append-only hash chain.

Durability and External Storage Anchoring

Receipt storage is hardened against both concurrent corruption and forensic tampering:

- **Single-Writer Transaction Isolation:** Local receipt state is managed via SQLite transactions executing compare-and-swap (CAS) checks on the chain head under full fsync durability (synchronous=FULL). If a storage outage occurs, the gateway halts fail-closed; it refuses to execute unrecorded physical actions.
- **External Evidence Anchoring:** Internal hash chains protect against reordering or middle deletion, but an attacker with storage-level access could theoretically delete trailing records and rewind the chain head. SASKI Agentic neutralizes this threat through periodic external anchoring. The local chain-head hash, cumulative receipt count, and prior anchor hash are signed via **Google Cloud KMS (Ed25519)** and published to **Google Cloud Firestore** under strict, append-only create() semantics. Any post-hoc truncation or rollback of the local receipt store creates a mathematical divergence that is immediately flagged during an independent audit.

7. Operationalizing Governance: Rulebooks and Gap Discovery

Deploying deterministic governance must not introduce operational paralysis. SASKI balances strict enforcement with operational agility through customer rulebook ownership and automated gap discovery.

The Three-Signature Rulebook Model

To prevent vendor liability capture and ensure strict separation of duties, operational rulebooks require a three-signature verification model before loading:

1. **Distribution Signature (SASKI Institute):** Cryptographically verifies the integrity of the underlying engine templates and distribution artifacts.
2. **Authoring Signature (Systems Integrator / Engineer):** Confirms the identity of the engineer who drafted or customized the rule declarations.

3. Approval Signature (Customer Domain Expert): Confirms that an authorized operational leader at the deploying organization has accepted the rulebook as binding policy. This step formally anchors governance accountability with the enterprise operator rather than the software vendor.

Automated Rulebook Gap Discovery

In production environments, autonomous agents frequently evolve new capabilities or attempt novel, benign tool patterns that were not anticipated during initial rulebook authoring. SASKI handles this through an automated **Gap Discovery Architecture**:

- **Taxonomic Split:** When a request is denied, the gateway deterministically categorizes the event as either a *Rule Violation* (an action that explicitly breached an authored rule) or a *Policy Gap* (an action targeting an undeclared tool or resource).
- **Structured Gap Store:** Policy gaps are written to an internal, structured store, capturing frequency, originating agent, and resource targets while maintaining links to signed receipts.
- **Frequency Burst Alerting:** If a specific policy gap crosses a configured frequency threshold across distinct sessions, the gateway fires an immediate alert via an operations sidecar (e.g., Slack). Operators can review aggregated gap reports, decide whether to expand policy, and promote validated rules through the standard signature workflow without taking the production system offline.

8. A Low-Risk Evaluation Path

Implementing deterministic action governance does not require replacing existing agent frameworks or disrupting live operations. SASKI provides a clear, three-phase onboarding path:

- 1. Local Shadow Replay:** Using SASKI Shadow Replay, engineering teams run historical agent execution traces and tool logs through the compliance engine entirely within their local infrastructure. Tool arguments and conversation histories never leave the customer's secure boundary. The engine outputs an aggregate audit report identifying historical boundary violations, parameter overflows, and unmonitored tool calls.
- 2. Passive Gateway Proxy (Shadow Mode):** The gateway is inserted as a transparent MCP proxy between the agent orchestrator and downstream tools. It evaluates live traffic against a starter rulebook, logs verdicts, and tests sequence rules without interrupting live execution.
- 3. Active Enforcement:** Validated policies are flipped to active enforcement. The gateway enforces fail-closed execution, activates the human-in-the-loop hold queue for high-consequence operations, and emits cryptographically anchored receipt chains.

Conclusion: Governance as the Enabler of Autonomous Agency

The enterprise adoption of agentic AI will not be constrained by a lack of model intelligence; it will be constrained by the lack of enforceable boundaries. An enterprise cannot deploy an autonomous agent whose operational ceiling is defined only by the statistical probability that it will follow its prompt.

By separating cognitive planning from deterministic execution, organizations can unleash the reasoning power of foundation models while maintaining mathematically verifiable control over real-world actions. SASKI Agentic provides the essential transport-layer middleware that makes autonomous agents safe, compliant, and defensible in the enterprise.

References

- Bajaj, P. (2026). *Evaluation Blindness: How Silent Measurement Failures Corrupt AI Systems from Training to Deployment*. Independent Researcher. arXiv:2608.02786.
- Khan, S. (2026). *Verified Detection and Prevention of Concurrency Anomalies in Multi-Agent Large Language Model Systems*. University of the West of England, Bristol. arXiv:2606.17182.
- Li, F., Zhang, A., and Lv, C. (2026). *Constraint Tax in Open-Weight LLMs: An Empirical Study of Tool Calling Suppression Under Structured Output Constraints*. arXiv:2606.25605.
- Mishra, S., Chukkapalli, D., and Naik, G. R. (2026). *Stateful Guardrails for Multi-Turn LLM Systems: A Conversational Risk Accumulation Framework*. arXiv:2607.19361.
- Rashie, D., and Rashi, V. (2026). *Type-Checked Compliance: Deterministic Guardrails for Agentic Financial Systems Using Lean 4 Theorem Proving*. arXiv:2604.01483.
- Schroeder, K., and Wood-Doughty, Z. (2024). *Can You Trust LLM Judgments? Reliability of LLM-as-a-Judge*. Northwestern University. arXiv:2412.12509.
- Wang, C., and Zheng, Y. (2026). *Sandlock: Confining AI Agent Code with Unprivileged Linux Primitives*. arXiv:2605.26298.
- Wu, B., Zhang, W., Chen, K., Fang, H., and Yu, N. (2026). *Provably Secure Agent Guardrail*. University of Science and Technology of China. arXiv:2605.29251.
- Li, J.-J., He, J., Shang, C., Kulshreshtha, D., Xian, X., Zhang, Y., Su, H., Swamy, S., and Qi, Y. (2025). *STAC: When Innocent Tools Form Dangerous Chains for LLM Agents*. arXiv:2509.25624.